

Проверка на ацикличность

В случае **ориентированного графа** произведём серию обходов в глубину.

То есть из каждой вершины, в которую мы ещё ни разу не приходили, запустим поиск в глубину, который при входе в вершину будет красить её в серый цвет, а при выходе из нее – в чёрный. И, если алгоритм пытается пойти в серую вершину, то это означает, что цикл найден.

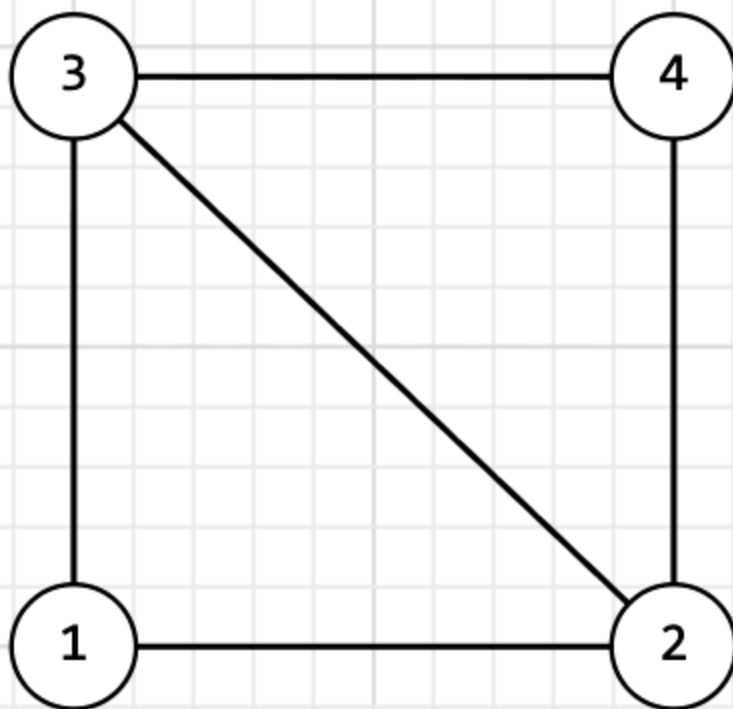
В случае **неориентированного графа**, необходимо дополнительно проверять, что текущее рассматриваемое из вершины ребро не является тем ребром, по которому мы пришли в эту вершину.

Для **восстановления** самого цикла достаточно при запуске поиска в глубину из очередной вершины добавлять эту вершину в стек. Когда поиск в глубину нашел вершину, которая лежит в цикле, будем последовательно вынимать вершины из стека, пока не встретим найденную еще раз.

Реализация для случая ориентированного графа

```
1  // color – массив цветов, изначально все вершины белые
2  void dfs(v) {
3      color[v] = grey;
4      for u in Adj[v] do // Adj – список смежности вершины v
5          if (color[u] == white)
6              dfs(u)
7          if (color[u] == grey)
8              circle = true
9      color[v] = black
10 }
11 for v in V do
12     color[v] = white;
13 circle = false
14 for v in V do
15     if color[v] == white
16         dfs(v)
```

Можно ли найти все циклы? Смысла искать все циклы нет. Например на рисунке 3 цикла, но достаточно найти 2, а 3 построить на их основе.



Вводим понятие фундаментальных циклов графа – это множество таких циклов, что с помощью их линейной комбинации можно выразить все циклы.

Алгоритм. (Нахождение множества элементарных циклов графа)

Данные: Граф $G = (V, E)$, представленный списками инцидентности $ЗАПИСЬ[v]$,

Результаты: Множество элементарных циклов графа G .

```

1  procedure ЦИКЛ(u): (* нахождение фундаментального множества циклов для
   компонент связности, содержащей вершину u, переменные d, num, СТЕК,
   ЗАПИСЬ, WGN – глобальные*)
2      begin d := d + 1; СТЕК[d] := v; num := num + 1; WGN[v] := num;
3          for u in ЗАПИСЬ[v] do
4              if WGN[u] = 0 then ЦИКЛ(u)
5              else if (u != СТЕК[d - 1]) and (WGN[v] > WGN[u]) then (* {v,
   u} замыкает новый цикл *)
6                  выписать цикл с вершинами
7                  СТЕК[d], СТЕК[d - 1], ..., СТЕК[e], где СТЕК[e] = u;
8                  d := d - 1 (* использованная вершина v удаляется из стека *)
9          end; (* ЦИКЛ *)
10     begin (* главная программа *)
11         for v in V do WGN[v] := 0; num := 0; (* инициализация *)
12         d := 0; СТЕК[0] := 0; (* d = число элементов в стеке *)
13         for v in V do
14             if WGN[v] = 0 then ЦИКЛ(u)

```

Оценим теперь вычислительную сложность этого алгоритма. Отметим сначала, что общее число шагов, не считая выписывания циклов (строки 6, 7) — как и во всех алгоритмах, основанных на поиске в глубину, — имеет порядок $O(n + m)$. К этому следует прибавить суммарную длину всех циклов. Эта длина не превосходит $(m - n + 1)$, что дает общую сложность алгоритма $O(nm + n)$, (или $O(nm)$, если отбросить вырожденный случай $m = 0$).

Проверка графа на двудольность

На основе обхода в глубину

На каждом шаге обхода в глубину помечаем вершину.

Допустим, мы пошли в первую вершину — помечаем её как 1. Затем просматриваем все смежные вершины, и если не помечена вершина, то на ней ставим пометку 2 и рекурсивно переходим в нее. Если же она помечена и на ней стоит та же пометка, что и у той, из которой шли (в нашем случае 1), значит граф не двудольный.

```
1  void dfs(v)
2      for u in Adj[v] do
3          if color[u] == 0
4              color[u] = 3 - color[v]
5              dfs[u]
6          else if color[u] == color[v]
7              flag = false
8
9  for (v in V) do
10     color[v] = 0;
11     flag = true
12     for v in V do
13         if color[v] == 0
14             color[v] = 1
15             dfs[v]
```

На основе обхода в ширину

Произведём серию поисков в ширину. Т.е. будем запускать поиск в ширину из каждой непосещённой вершины. Ту вершину, из которой мы начинаем идти, мы помещаем в первую долю. В процессе поиска в ширину, если мы идём в какую-то новую вершину, то мы помещаем её в долю, отличную от доли текущей вершины. Если же мы пытаемся пройти по ребру в вершину, которая уже посещена, то мы проверяем, чтобы эта вершина и текущая вершина находились в разных долях. В противном случае граф двудольным не

является. По окончании работы алгоритма мы либо обнаружим, что граф не двудолен, либо найдём разбиение вершин графа на две доли.

Поиск кратчайших путей во взвешенном графе

Граф называется **взвешенным**, если каждому его ребру (дуге) приписано некоторое вещественное число, называемое весом данного ребра (дуги).

Кратчайший путь – это путь минимального веса.

Вес пути – это сумма весов, входящих в него ребер (дуг).

Существует множество прикладных задач, которые можно решать с помощью данных алгоритмов, причем не обязательно вес – это именно длина пути в км (или других единицах измерения длин). Это может быть и цена бензина, цена билета (на поезд или самолет) или вообще любая стоимость проезда и т.д.

В основном все известные алгоритмы решают один из двух вариантов задачи:

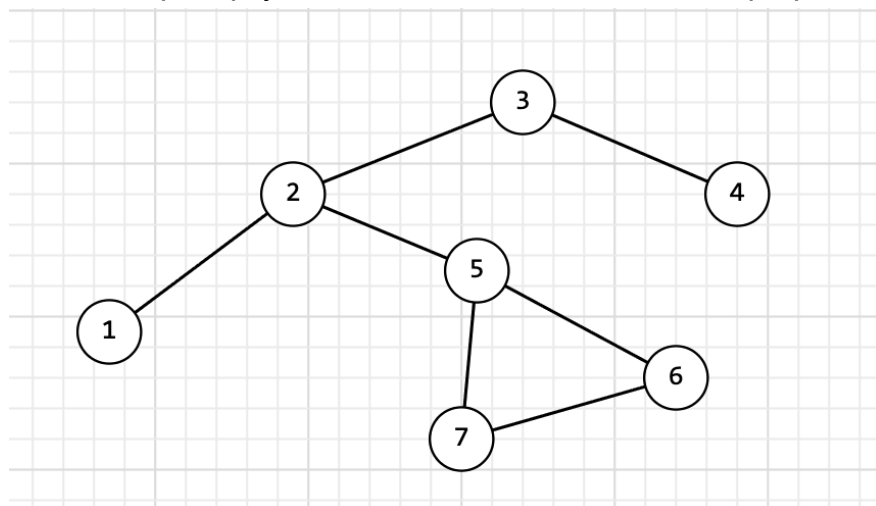
1. поиск кратчайшего пути от заданной вершины до всех остальных (алгоритм Форда, Дейкстры, алгоритм поиска кратчайших путей в ориентированном ациклическом графе)
2. поиск кратчайшего пути между всеми парами вершин (алгоритм Флойда, алгоритм Джонсона)

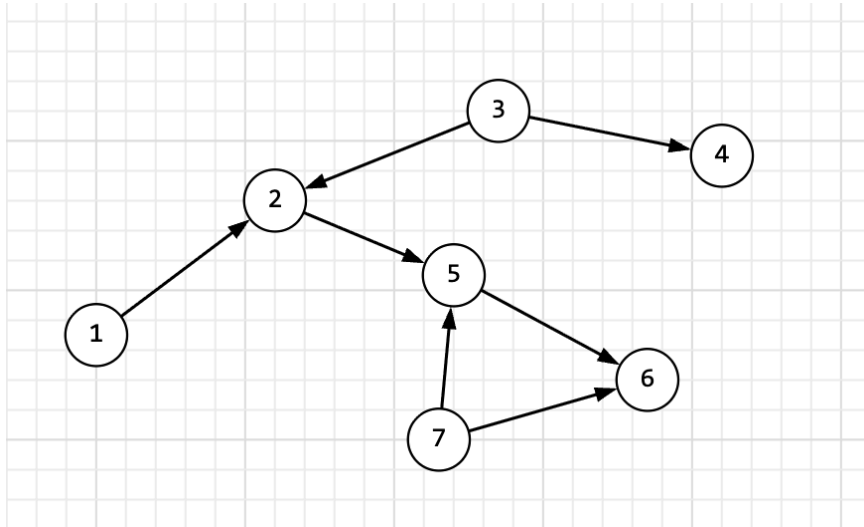
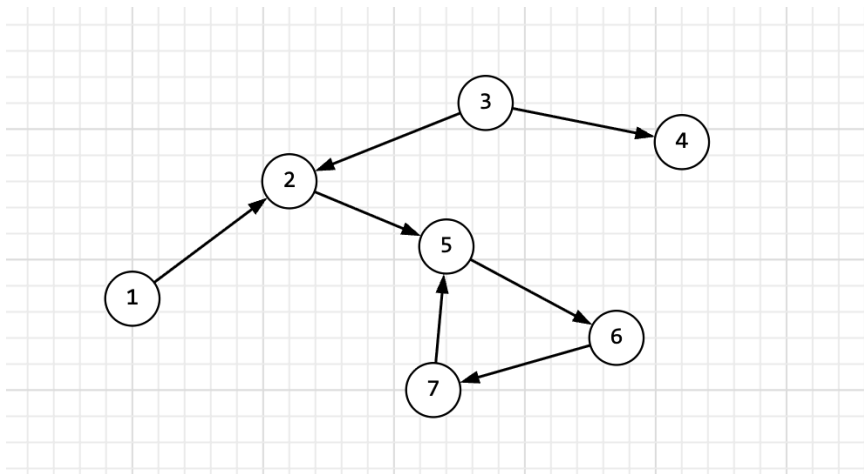
Кроме того, у этих алгоритмов могут различаться требования к графу, с которым они работают и время работы. Например, алгоритм Дейкстры работает только с графами, у которых все веса положительны.

Еще на выбор алгоритма может повлиять такое свойство графа как **разреженность** (например, на разреженных графах алгоритм Джонсона будет работать быстрее, чем алгоритм Флойда)

Д/з

1. Сделать проверку на ацикличность по шагам на графах.





2. Написать алгоритм нахождения множества элементарных циклов графа. (на хорошую оценку)

